

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



**WEB APPLICATION DEVELOPMENT
PROJECT REPORT
RESTAURANT MANAGEMENT SYSTEM
GROUP 12 - MEMBER LIST**

- | | | |
|--------------------------|-------------|-------------|
| 1. Vũ Đức Nhân | ITCSIU24063 | Team Leader |
| 2. Nguyễn Triều Hưng | ITCSIU24031 | Team Member |
| 3. Phạm Như Nguyên Trung | ITCSIU24091 | Team Member |

TABLE OF CONTENTS

CHAPTER 1 EXECUTIVE SUMMARY.....	1
CHAPTER 2 REQUIREMENT ANALYSIS.....	2
2.1. Problem Statement.....	2
2.2. User Stories.....	3
2.3. Functional Requirements.....	7
2.3.1. Customer Portal.....	7
2.3.2. Admin Portal.....	9
2.3.3. Shipper Portal.....	10
2.4. Non-Functional Requirements.....	11
2.4.1. Performance.....	11
2.4.2. Security.....	11
2.4.3. Usability.....	12
2.4.4. Scalability.....	13
2.4.5. Reliability & Availability.....	13
CHAPTER 3 SYSTEM ARCHITECTURE.....	14
3.1. Architecture Overview.....	14
3.2. Technology Stack Justification.....	15
3.3. Design Patterns.....	17
3.4. API Design.....	18
CHAPTER 4 DATABASE DESIGN.....	21
4.1. Entity.....	21
4.2. Table Schemas.....	25
4.3. Relationships Explanation.....	30
4.4. Normalization Analysis.....	31
4.5. Indexing Strategy.....	32
CHAPTER 5 IMPLEMENTATION DETAILS.....	35
5.1. Key Algorithms and Business Logic.....	35
5.2. Security Implementation.....	35
5.3. Error Handling Strategy.....	35
CHAPTER 6 TESTING AND QUALITY ASSURANCE.....	36
6.1. Testing Strategy.....	36

6.2. Bug Tracking.....	40
6.3. Known Limitations.....	41
CHAPTER 7 DEPLOYMENT.....	42
7.1. Deployment Process.....	42
7.2. Monitoring and Maintenance.....	42
CHAPTER 8 USER GUIDE.....	43
8.1. Getting Started.....	43
8.2. Feature Walkthroughs.....	44
8.3. Known Limitations.....	48
CHAPTER 9 TEAM COLLABORATION.....	51
9.1. Team Organization.....	51
9.2. Individual Contributions.....	51
9.3. Challenges in Teamwork.....	51
CHAPTER 10 CONCLUSION AND REFLECTION.....	52
10.1. Project Summary.....	52
10.2. Lessons Learned.....	52
10.3. Individual Reflections.....	53
10.4. Future Enhancements.....	53
REFERENCES.....	49

LIST OF FIGURES

Figure 4.1:	ER
Diagram.....	12
Figure 4.2:	Table
Schema.....	16

CHAPTER 1

EXECUTIVE SUMMARY

Project name: VNFood Restaurant Management System

Team: 12

Key features

Customer Features:

- User Authentication — Register / Login with email & password, role-based redirect (customer vs shipper)
- Menu Browsing — Category filtering (Phở & Bún, Cơm, Bánh Mì, etc.), food cards with quick view modal
- Shopping Cart — Add/remove items, adjust quantity, persistent via localStorage, price formatting in VND
- Checkout & Payment — Delivery address, phone, cash or card payment, order total with delivery fee & discount
- Order Tracking (Billing) — View order history, track status progress (Xác Nhận → Chuẩn Bị → Kiểm Tra → Giao → Đã Giao), cancel orders
- Receipt / Invoice — Detailed receipt view per order with item breakdown and payment info
- Table Booking — Reserve tables with name, phone, date/time, number of people
- Promotions Page — Static promotional offers display
- About Page — Restaurant information

Admin Features:

- Admin Login — Shared password-based login (bcrypt hash via .env)
- Order Management Dashboard — View all orders, advance/change status, mark paid/unpaid, view order details
- Dish Management — CRUD dishes with image upload
- Shipper Account Management — Create shipper accounts (email, name, password), view all shippers
- Revenue Summary — Total bills, total/paid/unpaid revenue metrics

Shipper Features:

- Shipper Login — Via the regular login page (auto-redirects based on role)
- Available Orders — View unassigned orders with polling (every 5s)
- Accept/Deny Orders — One order at a time enforcement

- Delivery Tracking — Progress bar (Confirmed → Preparing → Checking → Delivering → Delivered)
- Pickup & Delivery — "Lấy Hàng" when food is ready, "Đã Giao" when delivered, auto status updates

Technical Stack:

- Frontend — React 19, Vite, Tailwind CSS, React Router, Axios
- Backend — NestJS (TypeScript), TypeORM, MySQL
- Database — 6 tables: users (with role), foods, cart_items, bookings, bill_status (with shipper assignment), bill_details
- 3 user roles — Customer (DB account), Shipper (DB account, role='shipper'), Admin (env password, no DB record)

Technology stack summary

Frontend

Technology	Version	Purpose
React	19.2.6	UI framework
Vite	8.0.12	Build tool & dev server
Tailwind CSS	4.3.0	Styling
React Router DOM	7.1.5	Client-side routing
Axios	1.7.9	HTTP client for API calls

Backend

Technology	Version	Purpose
NestJS	11.0.1	Server framework
TypeScript	5.7.3	Language
TypeORM	1.0.0	ORM / database queries
MySQL2	3.22.3	Database driver
bcryptjs	2.4.3	Password hashing

Database

MySQL: relational database with 6 tables

- **users, foods**
- **cart_items**
- **bookings**
- **bill_status**
- **bill_details**

Deployment URL and test credentials

- Deployment URL:
<https://successful-reverence-production-939c.up.railway.app/>
- Test credentials

Role	Email / Access	Password	Login URL
Customer	test@gmail.com	test1234	/login
Shipper	shipper1@gmail.com	123456	/login
Admin	(password only)	password123	/admin

CHAPTER 2 REQUIREMENT ANALYSIS

2.1. Problem statement

Traditional restaurant ordering and delivery processes often rely on channels such as phone calls, messages or manual records. This can lead to incorrect orders, unclear delivery progress and inefficient coordination between the restaurant and delivery.

VNFood Restaurant Ordering System provides a web platform where customers can place food orders and book tables, administrators can manage restaurant operations, and shippers can handle deliveries through a clear order workflow

Target Users and Pain Points

User	Pain points	Solution
Customer	Difficult to browse dishes, place orders and track delivery progress	Online menu, cart, checkout, table booking

		and order tracking
Admin	Managing dishes, orders, bookings and shippers manually is time-consuming	Centralized dashboard for restaurant management
Shipper	Difficult to find available orders and know when food is ready for pickup	Shipper dashboard with available orders and status updates

2.2. User stories

Must have:

#	Role	User Story	Feature Area
1	Customer	As a customer, I want to register an account with my email, name, and password, so that I can place orders and track my history.	Authentication
2	Customer	As a customer, I want to log in with my email and password, so that I can access my account and order food.	Authentication
3	Customer	As a customer, I want to browse the menu with category filtering (Phở & Bún, Com, Bánh Mì, etc.), so that I can quickly find the type of food I want.	Menu
4	Customer	As a customer, I want to add items to my cart and adjust quantities, so that I can build my order before checking out.	Cart
5	Customer	As a customer, I want to checkout with my delivery address, phone number, and payment method , so that the restaurant knows where and how to deliver my order.	Checkout
6	Customer	As a customer, I want to view my order history and track each order's	Billing / Order Tracking

		status (Confirmed → Preparing → Checking → Delivering → Delivered), so that I know when my food will arrive.	
7	Customer	As a customer, I want to cancel my order before it is picked up for delivery , so that I can change my mind without being charged.	Billing
8	Customer	As a customer, I want to view a detailed receipt for each order showing items, quantities, prices, discounts, and delivery fees, so that I can verify my charges.	Receipt
9	Admin	As an admin, I want to log in with a secure password , so that only authorized staff can access the management dashboard.	Admin Auth
10	Admin	As an admin, I want to view all customer orders and advance their status through the preparation pipeline, so that the kitchen and delivery flow stays organized.	Order Management
11	Admin	As an admin, I want to add, edit, and manage dishes (name, price, description, image, category), so that the menu stays up to date.	Dish Management
12	Admin	As an admin, I want to create shipper accounts (email, name, password), so that delivery personnel can log in and accept orders.	Shipper Management
13	Shipper	As a shipper, I want to log in with my credentials and be automatically redirected to the shipper dashboard, so	Shipper Auth

		that I can start accepting deliveries.	
14	Shipper	As a shipper, I want to see a list of available orders that need delivery, so that I can choose which order to take.	Shipper Dashboard
15	Shipper	As a shipper, I want to accept an order and have it assigned exclusively to me, so that no other shipper picks up the same order.	Order Assignment
16	Shipper	As a shipper, I want to be notified when the food is ready for pickup (status = Checking), so that I know when to go to the restaurant.	Delivery Tracking
17	Shipper	As a shipper, I want to mark an order as "Lấy Hàng" (Picked Up) , which automatically updates the status to "Đang Giao", so that the customer knows their food is on the way.	Delivery Status
18	Shipper	As a shipper, I want to mark an order as "Đã Giao" (Delivered) , so that the order is completed and I can accept a new one.	Delivery Completion

Should have:

#	Role	User Story	Feature Area
19	Customer	As a customer, I want to quick-view a dish's details (description, price, rating) in a modal without leaving the menu page, so that I can decide faster.	Menu UX
20	Customer	As a customer, I want my cart to persist across browser sessions , so that I don't lose my selections if I close the tab.	Cart Persistence

21	Customer	As a customer, I want to book a table by providing my name, phone, date, time, number of people, and optional notes, so that I can reserve seating at the restaurant.	Table Booking
22	Admin	As an admin, I want to see a revenue summary (total bills, paid/unpaid revenue, average bill), so that I can monitor the business at a glance.	Dashboard Analytics
23	Admin	As an admin, I want to view a list of all registered shippers , so that I can manage the delivery team.	Shipper Management
24	Shipper	As a shipper, I want to deny/release an accepted order back to the available pool (before pickup), so that another shipper can take it if I'm unable to deliver.	Order Flexibility

Nice to have:

#	Role	User Story	Feature Area
25	Customer	As a customer, I want to view promotions and special offers , so that I can take advantage of discounts.	Promotions

2.3. Functional requirements

2.3.1 Customer Portal

- **FR-CUST-01: User Registration**

- The system must allow new users to register an account by providing their Full Name, Email Address, and Password.
- The email address must be unique; registration must fail if the email is already in use.

- **FR-CUST-02: User Authentication**

- Registered customers must be able to log in securely using their Email and Password.
- The system must persist the login state in local storage to keep the session active across page reloads.
- **FR-CUST-03: Menu Browsing**
 - Customers must be able to browse the complete restaurant menu showing dish name, price, description, and image.
 - Customers must be able to filter dishes by category (e.g., Phở & Bún, Com, Bánh Mi, Drink, etc.).
- **FR-CUST-04: Dish Detail Quick-View**
 - The system must provide a Quick-View modal when a dish is selected, displaying detailed descriptions, larger images, pricing, and an "Add to Cart" option.
- **FR-CUST-05: Shopping Cart Management**
 - Customers must be able to add, update (increment/decrement), and delete items in their cart.
 - The cart must persist dynamically using localStorage so that items are retained if the user leaves or closes the browser tab.
- **FR-CUST-06: Checkout & Order Placement**
 - Customers must be able to place an order by providing a Delivery Address, Phone Number, and selecting a Payment Method (e.g., Cash on Delivery, Bank Transfer).
 - The system must calculate and display a breakdown of the subtotal, applied discounts, delivery fee, and final total before the order is finalized.
- **FR-CUST-07: Order Tracking & Status**
 - Customers must be able to view their order status history in real-time.
 - The order tracking interface must represent the linear status flow: CONFIRMED → PREPARING → CHECKING → DELIVERING → DELIVERED → COMPLETED.
- **FR-CUST-08: Order Cancellation**
 - Customers must have the option to cancel an order from their order history panel, provided the order status has not advanced past the CONFIRMED stage.
- **FR-CUST-09: Digital Receipts**
 - The system must generate a digital receipt for each completed or past order, detailing itemized prices, total cost, transaction date, and payment status.

- **FR-CUST-10: Table Booking / Reservation**
 - Customers must be able to request a table reservation by specifying Name, Phone, Date, Time, Number of People, Table Type, and optional notes.
 - The system must allow both authenticated users and guests to create a reservation.
- **FR-CUST-11: Promotions View**
 - Customers must be able to view current promotional offers and discounts via a dedicated Promotions page.

2.3.2 Admin Portal

- **FR-ADM-01: Admin Authentication**
 - Admins must be able to log in to the admin panel using a secure password verified against a server-side environment configuration.
- **FR-ADM-02: Dashboard Metrics & Revenue Summary**
 - The dashboard must display high-level metrics including: total revenue, total order counts, paid/unpaid statuses, and visual breakdown of orders by status.
- **FR-ADM-03: Order Pipeline Management**
 - Admins must be able to view all active and past orders from all users in a list/table.
 - Admins must be able to manually transition order statuses through the kitchen pipeline (e.g., from CONFIRMED to PREPARING, and from PREPARING to CHECKING).
- **FR-ADM-04: Dish Inventory (CRUD)**
 - Admins must be able to create new dishes, view existing dishes, update dish details (name, price, category, description), and delete dishes.
- **FR-ADM-05: Dish Image Upload**
 - The system must support uploading image files when creating or updating a dish.
 - Uploaded files must be securely saved to the backend disk and served dynamically via static routes.
- **FR-ADM-06: Shipper Account Creation**
 - Admins must have exclusive permission to create Shipper accounts by entering a Name, Email, and Password.
- **FR-ADM-07: Delivery Personnel Management**
 - Admins must be able to view a list of all registered shippers to track active delivery staff.

2.3.3 Shipper Portal

- **FR-SHIP-01: Automatic Role-Based Redirect**
 - If a user with the role shipper logs in using the standard login page, the system must detect their role and automatically redirect them to the Shipper Dashboard instead of the customer homepage.
- **FR-SHIP-02: Available Orders Board**
 - Shippers must see a list of orders in the CHECKING state that are ready for pickup and delivery.
 - This dashboard must poll the backend at regular intervals (5 seconds) to ensure shippers have the latest list of orders.
- **FR-SHIP-03: Order Acceptance**
 - Shippers must be able to accept an available order. Once accepted, the system must assign the order to that shipper (updating shipper_id in the database) and remove it from the public availability list.
 - Shippers can only handle one active delivery at a time.
- **FR-SHIP-04: Order Release / Denial**
 - Shippers must have the option to release/deny an accepted order before pickup, reverting its status back to CHECKING and returning it to the pool of available orders.
- **FR-SHIP-05: Picked Up Update ("Lấy Hàng")**
 - Shippers must be able to mark an order as picked up ("Lấy Hàng"), which updates the database order status to DELIVERING and notifies the customer that their food is on the way.
- **FR-SHIP-06: Delivery Completion ("Đã Giao")**
 - Shippers must be able to mark an order as delivered ("Đã Giao"), which updates the order status to DELIVERED, freeing the shipper to accept new orders.

2.4. Functional requirements

2.4.1 Performance

- **NFR-PERF-01: API Response Times**
 - Under normal load conditions, read operations (e.g., fetching the menu, loading order history) must respond within 300ms at the API layer.
 - Write operations (e.g., database writes for cart items, table bookings) must complete within 500ms.
- **NFR-PERF-02: Asset Delivery**

- Food images must be optimized and compressed to ensure load times do not degrade user experience. Images should be served with appropriate cache-control headers.
- **NFR-PERF-03: Frontend Hydration & Rendering**
 - The React application must leverage Vite's optimized build assets to achieve a Lighthouse Performance score of 90+ on desktop.
- **NFR-PERF-04: Polling Impact Mitigation**
 - Since the Shipper portal uses short polling (every 5 seconds) to fetch available orders, the backend query must be highly optimized, querying only `bill_status` records in the `CHECKING` state with no assigned shipper, to prevent database CPU spikes.

2.4.2 Security

- **NFR-SEC-01: Credential Protection**
 - All user passwords must be hashed using the `bcryptjs` algorithm with a minimum round cost of 10 before storage in the database. Raw passwords must never be stored, logged, or exposed in API responses.
- **NFR-SEC-02: Role-based Authorization**
 - The frontend must secure admin and shipper routes via client-side router checks to prevent unauthorized navigation.
 - The backend controllers must enforce security boundaries to ensure user accounts cannot modify administrative settings or access other users' cart items/order details.
- **NFR-SEC-03: CORS Configuration**
 - The backend NestJS API must implement a strict Cross-Origin Resource Sharing (CORS) policy, limiting requests strictly to authorized origins (e.g., the frontend dev/production domains).
- **NFR-SEC-04: Environment Configuration Protection**
 - All sensitive parameters, including database connection details, the admin password hash, and port configurations, must be stored in a `.env` file and never committed to source control.
- **NFR-SEC-05: Input Validation & Sanitization**
 - All incoming API request payloads must be validated using NestJS `ValidationPipe` combined with `class-validator` decorators to protect against SQL Injection, Cross-Site Scripting (XSS), and bad data payloads.

2.4.3 Usability

- **NFR-USA-01: Responsive Web Design**

- The frontend must be fully responsive, supporting mobile devices (320px width and up), tablets, and desktop resolutions using Tailwind CSS's responsive utility classes.
- **NFR-USA-02: User Experience (UX) Feedback**
 - All async actions (e.g., loading the menu, processing checkout, logging in) must display visual states such as loading spinners, skeleton UI screens, or button disabled states.
 - Successful and failed operations must display brief, floating alert notifications (toasts) or clear modal messages.
- **NFR-USA-03: Accessibility (a11y)**
 - The interface must follow Web Content Accessibility Guidelines (WCAG) 2.1 Level AA standards, specifically:
 - Ensuring text-to-background color contrast ratios of at least 4.5:1.
 - Providing descriptive alt tags on all food item images.
 - Ensuring keyboard navigability for interactive buttons and forms.

2.4.4 Scalability

- **NFR-SCAL-01: Concurrent Users**
 - The system must support a minimum of 100 concurrent active users (browsing, adding to cart, placing orders) without degradation in system performance or transaction failures.
- **NFR-SCAL-02: Database Connection Pooling**
 - The TypeORM connection pool must be configured to reuse connections efficiently, preventing database exhaustion from rapid API calls or polling.
- **NFR-SCAL-03: Stateless Backend Architecture**
 - The backend API must remain stateless. Since session storage is managed via localStorage on the client-side, the backend can scale horizontally by running multiple node processes behind a load balancer if needed.

2.4.5 Reliability & Availability

- **NFR-REL-01: System Uptime**
 - The application services (frontend and backend) must target a 99.5% uptime availability during standard operating hours.
- **NFR-REL-02: Transaction Integrity (ACID)**
 - Checkout operations must execute within a database transaction. If saving order status (BillStatus) succeeds but saving the line items (BillDetail) fails, the entire transaction must roll back to avoid orphaned or incomplete order records.

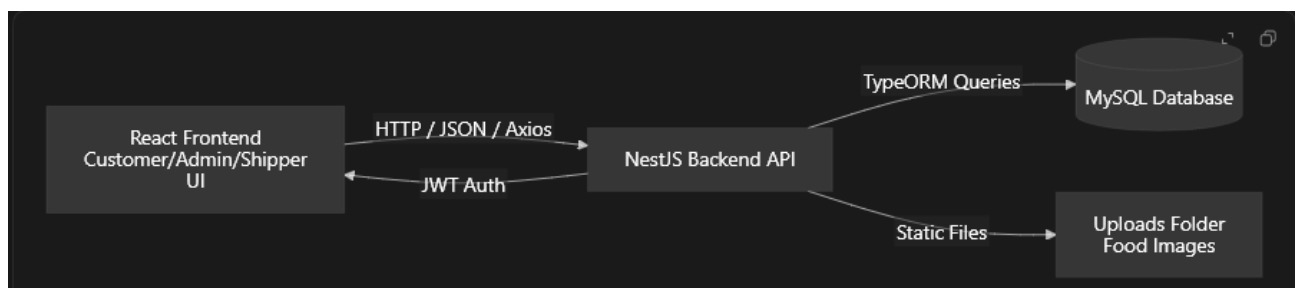
- **NFR-REL-03: Error Recovery & Logging**
 - The backend must implement global exception filters to catch unhandled errors, log details on the server for admin investigation, and return a clean, standard HTTP error payload (e.g., 500 Internal Server Error) to the client rather than exposing raw stack traces.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 Architecture Overview

The restaurant ordering system follows a three-tier client-server-database architecture. The frontend client provides the user interface for customers, administrators, and shippers. The backend server exposes HTTP APIs, handles authentication and authorization, applies business rules, and communicates with the database. The database stores persistent data such as users, foods, carts, bookings, bills, bill details, and order statuses.



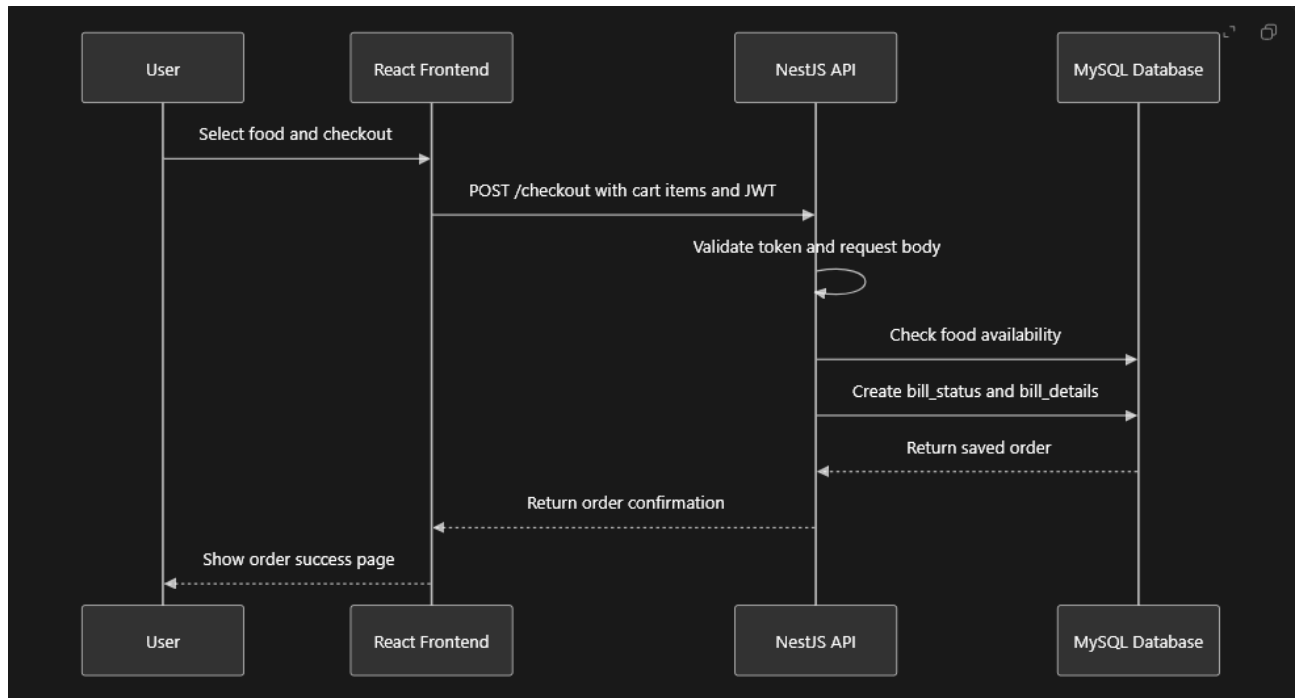
The **client layer** is implemented using React. It includes pages such as home, menu, cart, checkout, booking, login/register, admin dashboard, admin food management, admin bookings, billing, and shipper dashboard. The frontend communicates with the backend through Axios. Authentication tokens are stored in browser local storage and attached to protected API requests using the `Authorization: Bearer <token>` header.

The **server layer** is implemented using NestJS. It is divided into controllers, services, modules, DTOs, guards, and entity models. Controllers define API endpoints such as `/foods`, `/cart`, `/checkout`, `/bookings`, `/billing`, `/admin`, `/shipper`, `/login`, and `/register`. Services contain the business logic, such as validating checkout items, creating bills, checking table availability, managing food records, and updating order statuses. Guards enforce authentication and role-based authorization for admin, customer, and shipper features.

The **database layer** uses MySQL with TypeORM. The main tables include `users`, `foods`, `cart_items`, `bookings`, `bill_status`, and `bill_details`. Relational constraints are used to connect users with carts, bookings, orders, and bill details. This structure fits the system because restaurant ordering data is highly

relational: one user can have many orders, one order can contain many food items, and each bill detail belongs to a bill and a food item.

A typical order data flow is:



For administration, the flow is similar but protected by role-based access control. An admin logs in through `/admin/login`, receives a token, and can manage foods, bookings, shippers, and orders. Protected endpoints use `JwtAuthGuard` and `RolesGuard` to ensure that only authorized users can perform restricted actions.

Overall, the API design separates public endpoints from protected endpoints, uses JWT authentication for user identity, applies role checks for admin and shipper functions, and keeps resource operations predictable through REST-style routing.

3.2 Technology Stack Justification

The frontend uses **React with Vite**. React was chosen over vanilla JavaScript because the system contains many interactive screens and repeated UI patterns, such as food cards, modals, forms, navigation, carts, dashboards, and order status views. With vanilla JavaScript, DOM updates, state management, routing, and reusable UI would require more manual code and would be harder to maintain. React provides component-based development, making it easier to divide the interface into pages, layout components, context providers, and reusable UI components. Vite was chosen

because it provides fast development startup, hot module replacement, and a simple build process.

Compared with alternatives, React is more suitable than vanilla JavaScript for this project because the application is not a simple static website. It has authentication state, cart state, role-based screens, form handling, API loading states, and repeated components. Angular could also support this project, but it is heavier and more opinionated. Vue is simpler than React in some cases, but React has a larger ecosystem and strong support for SPA routing, hooks, context, and reusable components. Therefore, React provides the best balance between flexibility, maintainability, and development speed.

The backend uses **NestJS with TypeScript**. NestJS was chosen over raw Node.js/Express because it provides a structured architecture out of the box. Raw Express is flexible, but large applications can become difficult to organize when routes, middleware, validation, services, and database logic are not clearly separated. NestJS encourages a modular structure with controllers, providers, modules, guards, and dependency injection. This matches the needs of the restaurant ordering system because the backend has several domains: authentication, users, foods, cart, checkout, bookings, billing, admin, shipper, and audit logs.

If comparing to Java technologies, the same reasoning applies to the common statement: **“We chose Spring Boot over raw Servlets because Spring Boot provides built-in structure, dependency injection, configuration management, validation, and production-ready conventions, while raw Servlets require much more manual setup.”** In this actual project, NestJS plays a similar role in the Node.js ecosystem. It gives a structured framework instead of forcing the team to manually design every backend pattern from scratch.

The database uses **MySQL** with TypeORM. MySQL was chosen over MongoDB because the system data is relational and transactional. Orders, users, foods, bill details, bookings, and shippers have clear relationships. A relational database supports foreign keys, joins, constraints, and transactions, which are important for maintaining data consistency. For example, during checkout, the system must create an order record and multiple order detail records together. A relational model makes this process safer and easier to query.

MongoDB is useful for flexible document-based data, but this project benefits more from structured schemas. Restaurant ordering needs consistent fields such as user ID, food ID, quantity, price, order status, payment method, and booking date. Using MongoDB would require more manual handling of relationships and consistency. PostgreSQL would also be a strong alternative because it is a powerful relational database with advanced features. However, MySQL is widely supported, easy to deploy, familiar to many developers, and sufficient for the scale and requirements of this project.

TypeORM was selected because it connects NestJS entities directly to database tables. It reduces the amount of raw SQL needed in application code and provides repositories, query builders, relations, and transaction support. This improves maintainability while still allowing structured database access.

For hosting, the recommended deployment approach is to host the Railway. Vercel/Netlify are suitable for the frontend because React builds into static assets that can be served efficiently through a CDN. Render or Railway are suitable for the backend because they support Node.js applications, environment variables, build commands, and public HTTP services. A managed database is preferred over running MySQL manually because it simplifies backups, security, monitoring, and connection management. Our choice is Railway for frontend, backend and database because of it is beginner friendly.

3.3 Design Patterns

The system follows the **MVC pattern**. In this project, the React frontend acts as the View layer, while the NestJS backend separates Controllers and Models clearly. Controllers handle incoming HTTP requests and return responses. Services contain business logic. Entity classes represent database models. For example, `'FoodsController'` receives requests for food data, `'FoodsService'` performs the logic, and the `'Food'` entity maps to the `'foods'` table.

The **Repository pattern** is used through TypeORM repositories. Services inject repositories such as `'Repository<Food>'` or `'Repository<BillStatus>'` to access database records. This separates database operations from controller logic. Controllers do not directly write SQL or manipulate the database; they call services, and services use repositories. This improves testability, readability, and maintainability.

The **Singleton pattern** appears through NestJS dependency injection. Services such as authentication services, food services, billing services, and token services are registered as providers. NestJS manages their lifecycle and reuses provider instances where appropriate. This avoids manually creating service objects throughout the application and keeps shared logic centralized.

The system also uses the **Guard pattern** for security. ``JwtAuthGuard`` checks whether a request contains a valid bearer token, while ``RolesGuard`` checks whether the authenticated user has the required role. This keeps authentication and authorization separate from business logic. Instead of repeating role checks inside every controller method, the system applies guards declaratively.

The **DTO pattern** is used for structured request data. Create and update DTOs define expected fields for operations such as creating food, updating food, or creating bookings. Combined with validation pipes, DTOs help prevent invalid or unexpected request data from entering the business logic.

The **Mapper pattern** is used in response mapping functions. Database entities may contain internal field names or sensitive fields, while API responses should be clean and consistent. Response mappers transform database objects into frontend-friendly response objects.

3.4 API Design

The backend API follows RESTful principles by organizing endpoints around resources and actions. Main resources include foods, cart items, bookings, billing records, users, admin resources, and shipper orders. The current implementation uses routes such as ``/foods``, ``/cart/items``, ``/bookings``, ``/billing/invoice/:id``, and ``/admin/orders``. In a production version, these routes could be placed under a versioned prefix such as ``/api/v1``, for example ``/api/v1/foods/{id}``.

HTTP methods are used according to their purpose. ``GET`` retrieves data, such as food lists or booking records. ``POST`` creates new resources, such as registering a user, adding a cart item, creating a booking, or checking out. ``PATCH`` updates partial resource data, such as changing cart item quantity or updating an order status. ``PUT``

is used where a full update is appropriate, such as updating a food record. 'DELETE' removes or disables resources, such as removing cart items or deleting admin-managed records.

The API uses standard status code strategy. Successful reads return '200 OK'. Successful creation operations return '201 Created' where applicable. Invalid request data returns '400 Bad Request'. Missing or invalid authentication returns '401 Unauthorized'. Authenticated users without permission receive '403 Forbidden'. Missing records return '404 Not Found'. Unexpected server failures return '500 Internal Server Error'.

Example login request:

POST /login

Content-Type: application/json

```
{
  "email": "customer@example.com",
  "password": "password123"
}
```

Example login response:

```
{
  "user": {
    "id": 1,
    "email": "customer@example.com",
    "name": "Customer",
    "role": "customer"
  },
  "accessToken": "jwt-token-value"
}
```

Example food list request:

GET /foods?page=1&pageSize=10&category=Pho

Example checkout request:

POST /checkout

Authorization: Bearer jwt-token-value

Content-Type: application/json

```
{
  "phone": "0900000000",
  "address": "123 Main Street",
  "paymentMethod": "cash",
  "deliveryFee": 15000,
  "discount": 0,
  "total": 165000,
  "items": [
    {
      "foodId": 1,
      "quantity": 2,
      "price": 75000
    }
  ]
}
```

```
    }  
  ]  
}
```

Example checkout response:

```
{  
  "message": "Bill created",  
  "order": {  
    "id": 12,  
    "status": "confirmed",  
    "total": 165000,  
    "paymentMethod": "cash",  
    "paid": false  
  }  
}
```

Overall, the API design separates public endpoints from protected endpoints, uses JWT authentication for user identity, applies role checks for admin and shipper functions, and keeps resource operations predictable through REST-style routing.

CHAPTER 4

DATABASE DESIGN

4.1 Entity

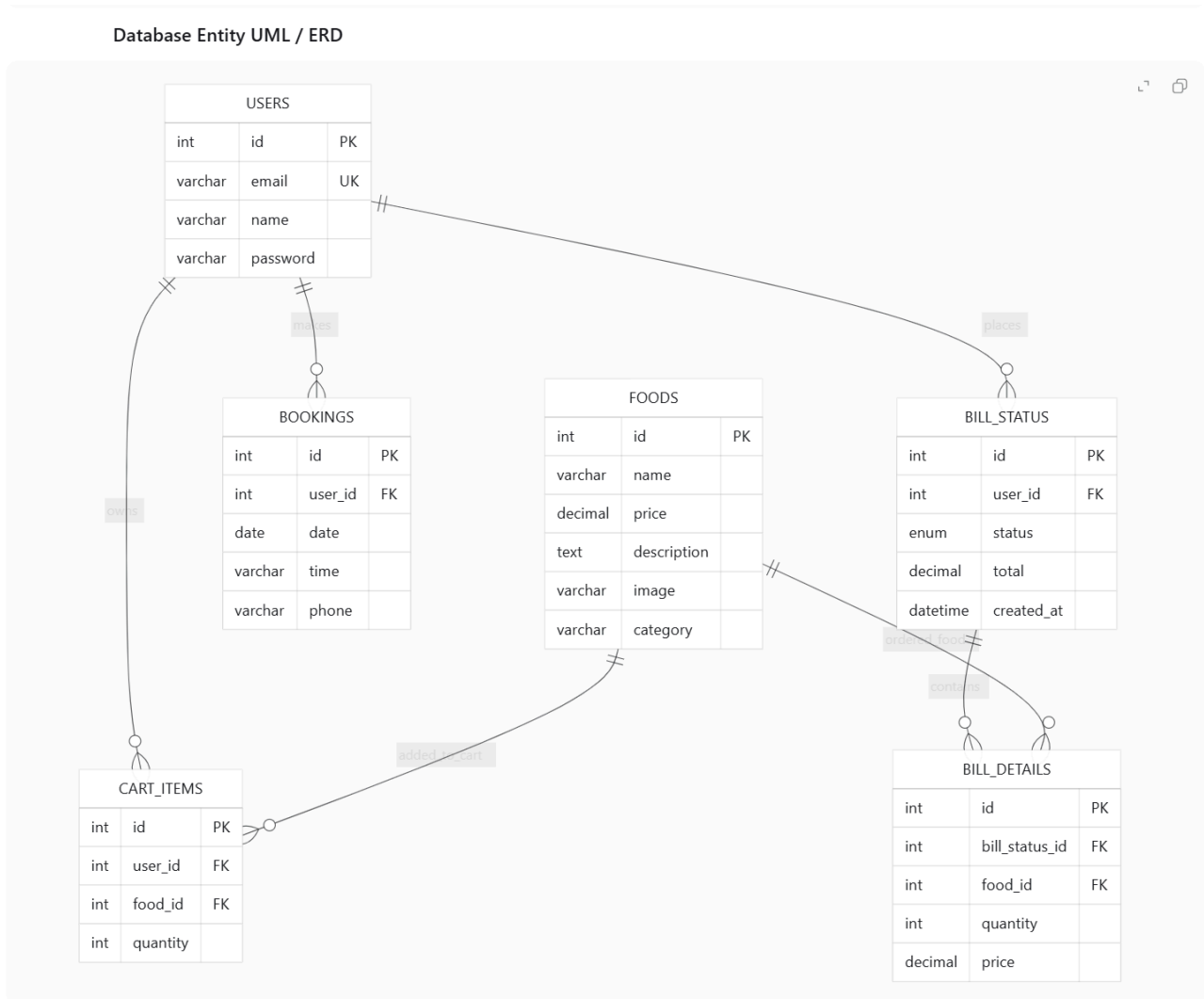


Figure 4.1. ER diagram

- **USERS**

- id (int) — PK (Primary Key)
- email (varchar) — UK (Unique Key)
- name (varchar)
- password (varchar)

- **BOOKINGS**

- id (int) — PK
- user_id (int) — FK (Foreign Key)

- date (date)
- time (varchar)
- phone (varchar)
- **CART_ITEMS**
 - id (int) — PK
 - user_id (int) — FK
 - food_id (int) — FK
 - quantity (int)
- **FOODS**
 - id (int) — PK
 - name (varchar)
 - price (decimal)
 - description (text)
 - image (varchar)
 - category (varchar)
- **BILL_STATUS**
 - id (int) — PK
 - user_id (int) — FK
 - status (enum)
 - total (decimal)
 - created_at (datetime)
- **BILL_DETAILS**
 - id (int) — PK
 - bill_status_id (int) — FK
 - food_id (int) — FK
 - quantity (int)
 - price (decimal)

Cardinalities and relationships

- **makes** (USERS to BOOKINGS):
 - **Cardinality:** One-to-Many (1:N)
 - **Details:** One USERS can make zero or many BOOKINGS (indicated by the zero-and-crow's-foot notation on the BOOKINGS side). One

BOOKINGS record belongs to exactly one USERS (indicated by the mandatory one/double-dash notation on the USERS side).

- **owns** (USERS to CART_ITEMS)
 - **Cardinality:** One-to-Many (1:N)
 - **Details:** One USERS can own zero or many CART_ITEMS. One CART_ITEMS record belongs to exactly one USERS.
- **places** (USERS to BILL_STATUS)
 - **Cardinality:** One-to-Many (1:N)
 - **Details:** One USERS can place zero or many bills in BILL_STATUS. One BILL_STATUS record belongs to exactly one USERS.
- **added_to_cart** (FOODS to CART_ITEMS)
 - **Cardinality:** One-to-Many (1:N)
 - **Details:** One FOODS item can be added to zero or many CART_ITEMS. One CART_ITEMS row links to exactly one mandatory FOODS
- **item.ordered_food** (FOODS to BILL_DETAILS)
 - **Cardinality:** One-to-Many (1:N)
 - **Details:** One FOODS item can appear in zero or many BILL_DETAILS line items. One BILL_DETAILS row links to exactly one mandatory FOODS
- **item.contains** (BILL_STATUS to BILL_DETAILS)
 - **Cardinality:** One-to-Many (1:N)
 - **Details:** One BILL_STATUS order can contain zero or many line items in BILL_DETAILS. One BILL_DETAILS record belongs to exactly one mandatory BILL_STATUS parent order.

4.2 Table Schemas

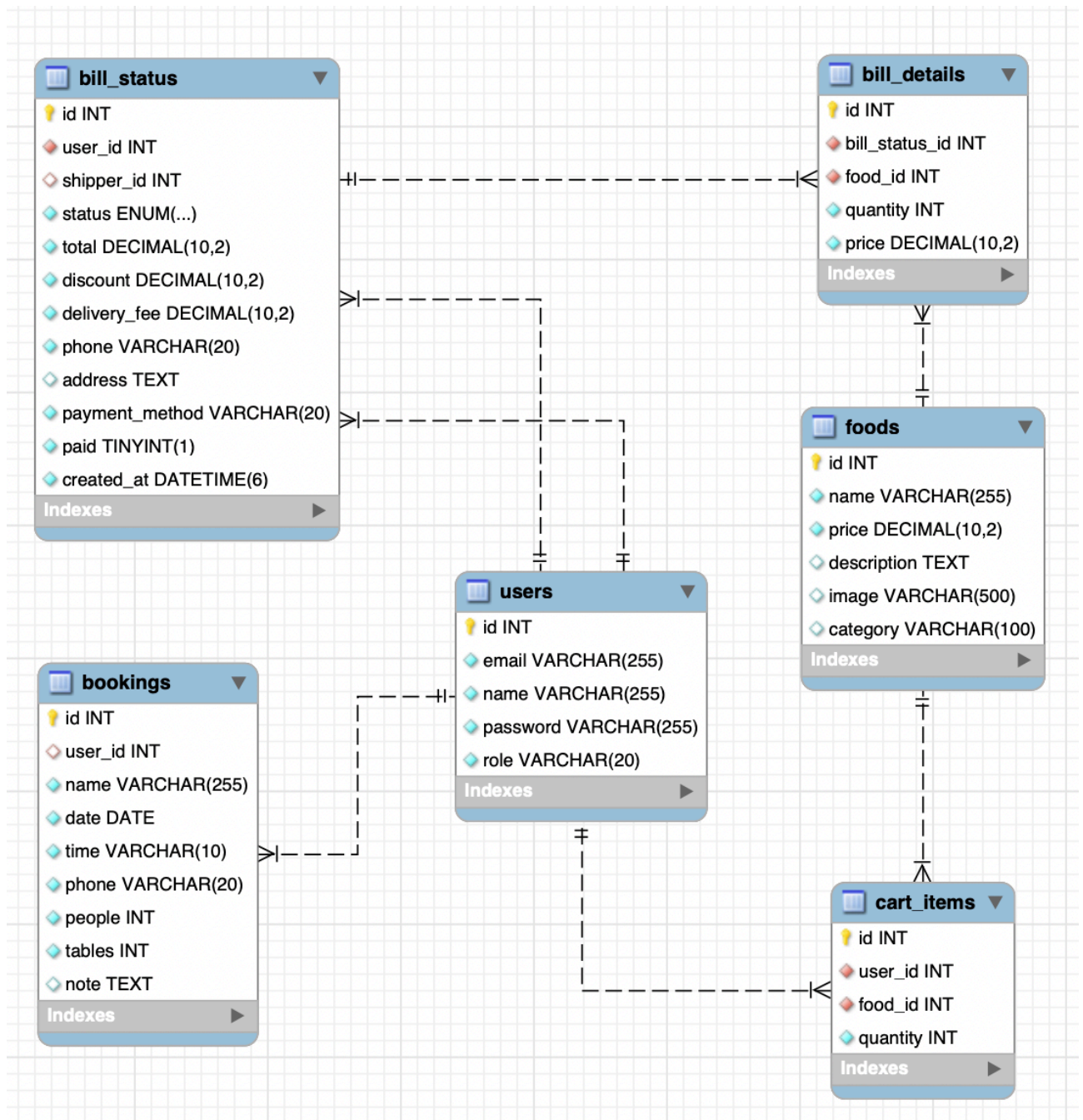


Figure 4.2. Table Schema

4.2.1 MySQL Code

Table: users

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  name VARCHAR(255) NOT NULL,  
  password VARCHAR(255) NOT NULL,
```

```
    role VARCHAR(20) NOT NULL DEFAULT 'customer'
);
```

Table: foods

```
CREATE TABLE foods (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    description TEXT NULL,
    image VARCHAR(500) NULL,
    category VARCHAR(100) NULL
);
```

Table: cart_items

```
CREATE TABLE cart_items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    food_id INT NOT NULL,
    quantity INT NOT NULL DEFAULT 1,
```

```
    CONSTRAINT fk_cart_items_user
    FOREIGN KEY (user_id)
    REFERENCES users(id)
    ON DELETE CASCADE,
```

```
    CONSTRAINT fk_cart_items_food
    FOREIGN KEY (food_id)
    REFERENCES foods(id)
    ON DELETE CASCADE
```

```
);
```

Table: bookings

```
CREATE TABLE bookings (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NULL,
    name VARCHAR(255) NOT NULL,
    date DATE NOT NULL,
    time VARCHAR(10) NOT NULL,
```

```
phone VARCHAR(20) NOT NULL,  
people INT NOT NULL DEFAULT 1,  
'tables' INT NOT NULL DEFAULT 1,  
note TEXT NULL,
```

```
CONSTRAINT fk_bookings_user  
    FOREIGN KEY (user_id)  
    REFERENCES users(id)  
    ON DELETE SET NULL  
);
```

Table: bill_status

```
CREATE TABLE bill_status (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL,  
    shipper_id INT NULL,  
    status ENUM(  
        'cancelled',  
        'pending',  
        'confirmed',  
        'preparing',  
        'checking',  
        'delivering',  
        'delivered',  
        'completed',  
        'paid'  
    ) NOT NULL DEFAULT 'confirmed',  
    total DECIMAL(10, 2) NOT NULL DEFAULT 0,  
    discount DECIMAL(10, 2) NOT NULL DEFAULT 0,  
    delivery_fee DECIMAL(10, 2) NOT NULL DEFAULT 0,  
    phone VARCHAR(20) NOT NULL DEFAULT "",  
    address TEXT NULL,  
    payment_method VARCHAR(20) NOT NULL DEFAULT 'cash',  
    paid TINYINT(1) NOT NULL DEFAULT 0,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    CONSTRAINT fk_bill_status_user  
        FOREIGN KEY (user_id)
```



```

REFERENCES users(id)
ON DELETE CASCADE,

CONSTRAINT fk_bill_status_shipper
FOREIGN KEY (shipper_id)
REFERENCES users(id)
ON DELETE SET NULL
);

```

Table: bill_details

```

CREATE TABLE bill_details (
  id INT AUTO_INCREMENT PRIMARY KEY,
  bill_status_id INT NOT NULL,
  food_id INT NOT NULL,
  quantity INT NOT NULL,
  price DECIMAL(10, 2) NOT NULL,

  CONSTRAINT fk_bill_details_bill_status
  FOREIGN KEY (bill_status_id)
  REFERENCES bill_status(id)
  ON DELETE CASCADE,

  CONSTRAINT fk_bill_details_food
  FOREIGN KEY (food_id)
  REFERENCES foods(id)
  ON DELETE CASCADE
);

```

4.3 Relationships Explanation

4.3.1 Relationships

users → bookings

- **Logic:** A registered client reserves a dining slot. The bookings table now includes operational details like the number of people, specific tables count, and a custom kitchen/seating note. A single user can hold many historical or future bookings, but each row in bookings references exactly one mandatory customer (user_id).

users → cart_items

- **Logic:** This handles the active, pre-checkout digital shopping cart state. A user can store multiple line items (cart_items) in their cart. Each row in cart_items maps exactly to one owner via user_id.

users → bill_status

- **Logic:** When a cart is processed, it generates a parent invoice header in bill_status. This table tracks financial metadata for an order (e.g., total, discount, delivery_fee, specific delivery address, payment_method, and a paid boolean flag). A user can place countless distinct orders over time, but each invoice is legally tied back to a single user_id.

bill_status → bill_details

- **Logic:** This represents the structural split between an order summary and its breakdown. bill_status holds the overall receipt details (like the final grand total and status), while bill_details acts as the breakdown lines showing what items were purchased. One bill has many detailed line entries; each line entry belongs uniquely to one parent bill_status_id.

foods → cart_items

- **Logic:** Menu items listed in foods can be simultaneously added to the active shopping baskets of multiple distinct browsing users. Each item in a cart points back to exactly one template food item to inherit its basic identity.

foods → bill_details

- **Logic:** A highly requested menu item will appear across hundreds of different archived receipts. When an item is bought, its active price is cloned straight into bill_details.price to lock down the transaction cost forever.

4.3.2 Junction Table

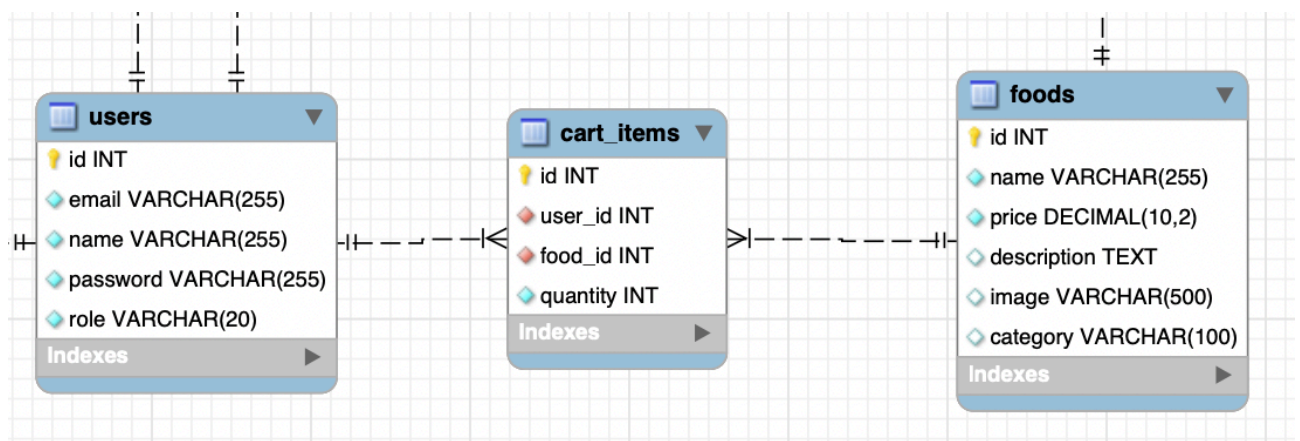


Figure 4.3.2. Junction table

CART_ITEMS and BILL_DETAILS function as junction tables.

4.3.3 Cascading Rules Justified

Rule 1: Use ON DELETE CASCADE for Carts and Booking Reservations

- **Application:** users(id) → cart_items(user_id) and users(id) → bookings(user_id).
- **Justification:** If an individual invokes their legal right to delete their application profile, keeping their abandoned un-purchased shopping cart rows or temporary dining table reservations inside your active system provides zero business value. Leaving them causes "orphan record memory leaks." Setting this to cascade ensures that dropping a user account automatically wipes out their active cart and bookings.

Rule 2: Use ON DELETE RESTRICT (or NO ACTION) for Invoices and Menus

- **Application:** users(id) → bill_status(user_id) and foods(id) → bill_details(food_id).
- **Justification: Crucial for accounting.** If a customer chooses to close their account, or if a restaurant stops cooking a particular menu dish, you cannot allow an ON DELETE CASCADE operation to execute. If it did, dropping a retired food item would automatically purge every historical line-item record of that food ever sold across your entire business history, completely breaking your historical accounting reports and tax ledgers.
- **The Standard Workaround:** Restricting the delete forces database administrators to use "Soft Deletion" (e.g., adding an is_active or archived column to the foods table) so the item can be hidden from frontend ordering screens while remaining perfectly preserved for backward-looking data analytics.

Rule 3: Use ON DELETE CASCADE for Order Component Line Items

- **Application:** bill_status(id) → bill_details(bill_status_id).
- **Justification:** The lines inside bill_details possess no independent meaning outside of the invoice document they belong to. If a system administrator explicitly purges an entire faulty order summary header from bill_status, leaving isolated fragments of food line-items around makes no structural sense. Dropping the parent invoice should immediately clean out its dependent child itemizations.

4.4 Normalization Analysis

4.4.1 Demonstration of 3NF

Example: Table `bill_status`.

- **id** is the Primary Key.
- **total, discount, delivery_fee, phone, address, payment_method,** and **created_at** are direct facts about that specific transaction instance (**id**).
- We do not store the customer's raw **name** or **email** in this table; instead, we use **user_id** as a reference. If we did store **user_name** here, it would create a transitive dependency ($\text{id} \rightarrow \text{user_id} \rightarrow \text{user_name}$), which would violate 3NF. Because we isolated user metadata to the **users** table, 3NF is perfectly preserved.

4.4.2 Functional Dependencies (FDs)

- **users:** $\text{id} \rightarrow \text{email, name, password, role}$ (Additionally, $\text{email} \rightarrow \text{id, name, password, role}$ because email is a Unique Key).
- **bookings:** $\text{id} \rightarrow \text{user_id, name, date, time, phone, people, tables, note}$.
- **foods:** $\text{id} \rightarrow \text{name, price, description, image, category}$.
- **cart_items:** $\text{id} \rightarrow \text{user_id, food_id, quantity}$.
- **bill_status:** $\text{id} \rightarrow \text{user_id, shipper_id, status, total, discount, delivery_fee, phone, address, payment_method, paid, created_at}$.
- **bill_details:** $\text{id} \rightarrow \text{bill_status_id, food_id, quantity, price}$.

4.5 Indexing Strategy

4.5.1 Indexed columns

- **Primary Keys (Clustered Indexes):** `users(id)`, `bookings(id)`, `cart_items(id)`, `foods(id)`, `bill_status(id)`, and `bill_details(id)`.
- **Foreign Keys (Secondary Indexes):** * `bookings(user_id)` via `FK_64cd97...`
 - `cart_items(user_id)` and `cart_items(food_id)`
 - `bill_status(user_id)` (Note: `shipper_id` is present but needs an explicit secondary index if frequently queried).
 - `bill_details(bill_status_id)` and `bill_details(food_id)`
- **Custom Unique/Secondary Indexes:** `users(email)` (denoted by `IDX_97672ac...`).

4.5.2 Why these indexes

- **Primary Key Indexes:** MySQL (using the InnoDB storage engine) automatically structures tables around a clustered index based on the Primary Key. This allows ultra-fast $O(1)$ lookups when retrieving specific single rows (e.g., fetching a specific dish by its ID).
- **Foreign Key Indexes:** When a user views their dashboard, the application executes queries like: `SELECT * FROM bill_status WHERE user_id = 42;` Without a secondary index on `user_id`, MySQL would have to perform a costly **Full Table Scan**, reading every single order row in the database just to find that user's history. Indexing foreign keys turns these into rapid index lookups.
- **The Email Index (IDX_97672ac...):** The email field is used during every single user authentication event. Indexing it ensures that verifying password hashes during login takes fractions of a millisecond.

4.5.3 Expected Query Performance Impact

Query Pattern Example	Impact Without Indexes	Impact With Current Indexes	Performance Gain
User Login (WHERE email = ?)	Full Table Scan (Reads all users)	B-Tree Index Seek	Instantaneous ($O(\log N)$)
Load Active Cart (WHERE user_id = ?)	Scans entire cart_items table	Directly targets user rows	High (Prevents bottlenecks during checkout)
Generate Invoice Breakdown (JOIN on bill_status_id)	Nested Loop Scan across millions of line items	Immediate pointer match	Critical (Prevents server timeouts on deep tables)

CHAPTER 5

IMPLEMENTATION DETAILS

5.1 Key Algorithms and Business Logic

1. Role-Based Login Routing

```
const handleSubmit = async (e) => {
  e.preventDefault();
  if (!validateForm()) return;
  setIsLoading(true);
  try {
    const user = await login(formData.email, formData.password);
    if (user?.role === 'shipper') {
      navigate('/shipper/dashboard');
    } else {
      navigate('/');
    }
  } catch (error) {
    if ([400, 401, 404].includes(error?.response?.status)) {
      setGeneralError('Sai email hoặc mật khẩu');
    } else {
      setGeneralError('Đăng nhập thất bại. Vui lòng thử lại.');
```

Why this is interesting: Instead of having separate login pages for customers and shippers (which was the original design), this code inspects the role field from the backend response and dynamically routes users to different dashboards from a single login form. This is a design pattern that scales well — if a new role like "chef" were added, it would only require one more else if branch. The error handling is also notable: it catches specific HTTP status codes (400, 401, 404) to show a user-friendly "wrong credentials" message, while falling back to a generic error for unexpected server issues (e.g., 500).

2. Shipper One-Order-at-a-Time Constraint

```
const ACTIVE_STATUSES = [
  BillStatusEnum.CONFIRMED,
  BillStatusEnum.PREPARING,
  BillStatusEnum.CHECKING,
  BillStatusEnum.DELIVERING,
];

async accept(billId: number, shipperId: number) {
  // Check if shipper already holds an active order
  const existing = await this.billStatusRepository.findOne({
    where: {
      shipperId,
      status: In(ACTIVE_STATUSES),
    },
  });
  if (existing) {
    throw new BadRequestException(
      'You already have an active order. Complete or deny it first.',
    );
  }

  const bill = await this.findEntity(billId);
  // Race condition guard: another shipper may have taken it
  if (bill.shipperId) {
    throw new BadRequestException('This order has already been taken by another shipper.');
```

Why this is interesting: This implements a three-layer validation gate before allowing a shipper to accept an order. First, it queries the database to check if the shipper already has any order in an "active" status (Confirmed through Delivering) — this enforces the business rule that shippers can only handle one delivery at a time, preventing a single shipper from hoarding orders. Second, it checks if another shipper has already claimed the order (a race condition guard for when two shippers click "Accept" at nearly the same time). Third, it validates the order's current status is still within the assignable range. This is essentially a state machine enforced at the service layer, where each transition has preconditions that must be met.

3. Cart Persistence with localStorage

```
const CART_STORAGE_KEY = 'qfood_cart';

export function CartProvider({ children }) {
  // Initialize state from localStorage (lazy initializer)
  const [items, setItems] = useState(() => {
    const saved = localStorage.getItem(CART_STORAGE_KEY);
    return saved ? JSON.parse(saved) : [];
  });

  // Sync to localStorage on every change
  useEffect(() => {
    localStorage.setItem(CART_STORAGE_KEY, JSON.stringify(items));
  }, [items]);

  const addItem = (food, quantity = 1) => {
    setItems(prev => {
      const existing = prev.find(item => item.foodId === food.food_id);
      if (existing) {
        // Merge: increment quantity instead of duplicating
        return prev.map(item =>
          item.foodId === food.food_id
            ? { ...item, quantity: item.quantity + quantity }
            : item
        );
      }
      // New item: extract only the fields we need
      return [...prev, {
        foodId: food.food_id,
        name: food.food_name,
        price: parseFloat(food.food_price),
        discount: parseFloat(food.food_discount || 0),
        image: food.food_image,
        quantity
      }];
    });
  };
}
```

Why this is interesting: The cart operates entirely without a backend. Instead, it uses React's `useState` with a lazy initializer (the arrow function in `useState`)

that reads from `localStorage` on the first render, and a `useEffect` that syncs back to `localStorage` whenever items changes. This creates a bi-directional persistence layer. The `addItem` function uses an important merge-or-create pattern: it checks if the item already exists in the cart and either increments the quantity (avoiding duplicates) or creates a new entry with only the necessary fields extracted from the full food object. The `parseFloat()` calls are a defensive measure because prices come from the database as strings.

5.2 Security Implementation

Passwords are protected using `bcryptjs`. When a user registers or changes password, the plaintext password is never stored directly.

```
const hashedPassword = await  
bcrypt.hash(createUserDto.password, 10);
```

The number 10 is the `bcrypt` cost factor. `Bcrypt` automatically generates a salt, so two users with the same password will still have different hashes. During login, the system compares the submitted password with the stored hash.

```
const isMatch = await bcrypt.compare(password, user.password);
```

SQL injection is reduced by using `TypeORM` repositories and parameterized `QueryBuilder` queries instead of manually joining SQL strings.

```
qb.andWhere(  
  '(LOWER(user.name) LIKE :search OR LOWER(user.email) LIKE  
:search)',  
  { search: `%${String(query.search).toLowerCase()}%` },  
);
```

The `:search` parameter is passed separately from the SQL structure, so user input is treated as data, not executable SQL.

XSS prevention is mainly handled by React's default escaping behavior. User values are rendered as JSX text instead of inserted as raw HTML. The project does not appear to use `dangerouslySetInnerHTML`, which reduces common reflected/stored XSS risk. However, because JWT tokens are stored in `localStorage`, strong XSS prevention is especially important.

Authentication uses a custom JWT service with HS256 HMAC signing. Tokens include user identity, role, issued time, and expiry time.

```
const payload = {  
  ...user,  
  iat: now,  
  exp: now + expiresInSeconds,  
};
```

The default token lifetime is 24 hours. The backend checks the bearer token in the Authorization header.

```
const [scheme, token] = authHeader.split(' ');  
  
if (scheme?.toLowerCase() !== 'bearer' || !token) {  
  throw new UnauthorizedException('Authentication required');  
}
```

Authorization is implemented using NestJS guards. RolesGuard checks whether the logged-in user has the required role, such as admin.

```
if (!user || !roles.includes(user.role)) {  
  throw new ForbiddenException('You do not have permission to  
access this resource');  
}
```

For user-owned resources, the system also checks ownership.

```
if (user.role === 'admin' || user.sub === resourceUserId) {  
  return;  
}  
  
throw new ForbiddenException('You can only access your own  
resources');
```

CSRF protection tokens are not currently implemented. Since the frontend sends JWTs in the Authorization header and withCredentials is disabled, the API is less exposed to traditional cookie-based CSRF. However, a future improvement would be to move JWTs to HTTP-only cookies and add CSRF tokens if cookie authentication is used.

5.3 Error Handling Strategy

On the backend, errors are handled using NestJS exceptions such as `BadRequestException`, `UnauthorizedException`, `ForbiddenException`, and `NotFoundException`. This gives the frontend consistent HTTP status codes.

Examples:

- Missing login fields return 400 Bad Request.
- Invalid password returns 401 Unauthorized.
- Accessing another user's bookings returns 403 Forbidden.
- Missing database records return 404 Not Found.

The global validation pipe also removes unexpected DTO fields and transforms request values.

```
app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
```

Database errors are reduced through TypeORM transactions and service-level validation. In checkout, if a food item is unavailable or bill detail creation fails, the transaction rolls back and avoids saving a partial order.

On the frontend, form errors are stored in local component state and displayed near the relevant fields. For example, checkout validates phone number, address, and card details before sending the request.

API-level errors are handled by an Axios response interceptor.

```
if (!isAuthEndpoint && [401, 403].includes(error.response?.status)) {  
  clearAuthStorage();  
  window.location.assign('/login');  
}
```

This prevents users from continuing with expired or invalid sessions. Individual pages also show user-friendly error messages, such as booking failure or checkout failure, instead of exposing raw backend details.

5.4 Advanced Features Implementation

The main advanced feature implemented in this project is real-time order status updates using WebSockets. Instead of requiring users, admins, and shippers to manually refresh pages, the system now broadcasts order changes immediately after important backend actions.

The backend uses NestJS WebSocket support with Socket.IO. A dedicated OrdersGateway handles WebSocket subscriptions, while RealtimeEventsService broadcasts order events after database updates. This separation keeps WebSocket logic reusable and avoids mixing socket code directly into every controller.

```
this.server.to('orders:admin').emit('order:changed', payload);  
  
this.server.to('orders:shippers').emit('order:changed',  
payload);  
  
this.server.to(`orders:user:${payload.order.user_id}`).emit('o  
rder:changed', payload);
```

This design allows different user roles to receive only relevant updates. Admins receive all order changes, customers receive updates for their own orders, and shippers receive updates when available or assigned orders change.

The frontend uses socket.io-client through a reusable subscribeToOrderEvents service. Pages such as the admin dashboard, billing page, and shipper dashboard subscribe to order:changed events and update their state immediately.

```
socket.on('order:changed', onOrderChanged);  
  
socket.emit('orders:subscribe', { userId });
```

The challenge was making real-time updates work without breaking the existing REST API flow. The solution was to keep REST endpoints as the source of database changes, then emit WebSocket events only after successful saves. This means the system still works normally if WebSocket connection fails, but users get a better live experience when it is connected.

The second advanced feature is role-based admin management. Admin-only endpoints use JWT authentication plus role checks. This protects

dashboards, booking management, food management, and user management from normal customers.

The third advanced feature is shipper order flow. Orders move through statuses such as confirmed, preparing, checking, delivering, and delivered. The shipper service prevents invalid transitions, such as picking up an order that is not ready or denying an order already being delivered.

The fourth advanced feature is cart and checkout persistence. The cart is stored in `localStorage`, so users do not lose selected food items when refreshing the page. During checkout, duplicate food items are normalized by food ID before saving, which keeps bill details cleaner.

5.5 AI Assistance Report

AI assistance was used during implementation to speed up development, debug problems, and improve code structure. The main areas where AI helped were backend service logic, security checks, frontend form handling, and report writing.

Another example was the JWT authentication flow. AI helped with the structure of signing and verifying tokens, including expiry checking and role information in the payload. The implementation was then adjusted to use the project's `AuthUser` type and existing guards.

AI also helped with frontend validation patterns. For example, checkout and booking pages use local error state, loading state, and clear submit handling. The generated validation logic was modified to match the application's Vietnamese user interface and project-specific fields.

AI-generated code was not accepted blindly. I reviewed the code, checked whether it matched the database entities, and changed parts that did not fit the project. For example, I made sure user IDs came from the authenticated token during checkout instead of trusting customer-submitted IDs. I also kept backend validation even when the frontend already performs checks, because frontend validation can be bypassed.

From using AI, I learned that AI is useful for producing first drafts and suggesting implementation patterns, but the developer still needs to verify security, database behavior, and project compatibility. The most important lesson was that AI output should be treated as assistance, not as a replacement for understanding the code.

CHAPTER 6

TESTING AND QUALITY ASSURANCE

6.1 Testing Strategy

User Authentication

ID	Test Scenario	Preconditions	Steps to Reproduce	Expected Result	Actual Result
TC-01	Test user registration with valid data	No existing account with the email	1. Navigate to /register 2. Enter name: "John Doe" 3. Enter email: john@gmail.com 4. Enter password: password123 5. Enter confirm password: password123 6. Click "Register"	Account is created successfully, user is redirected to the Login page with a success message	Account created, redirected to Login page
TC-02	Test user registration with duplicate email	Account with test@gmail.com already exists	1. Navigate to /register 2. Enter name: "Test User" 3. Enter email: test@gmail.com 4. Enter password: test1234 5. Enter confirm password: test1234 6. Click "Register"	Error message is displayed indicating the email is already registered	Error message "Email already exists" displayed
TC-03	Test user registration with mismatched passwords	None	1. Navigate to /register 2. Enter name: "Jane Doe" 3. Enter email: jane@gmail.com 4. Enter password:	Validation error: "Passwords do not match" is displayed, form is not	Error message displayed, registration blocked

			password123 5. Enter confirm password: different456 6. Click "Register"	submitted	
TC-04	Test user login with invalid credentials	Account with test@gmail.com exists	1. Navigate to /login 2. Enter email: test@gmail.com 3. Enter password: wrongpassword 4. Click "Login"	Error message is displayed indicating invalid email or password	Error message "Invalid email or password" displayed
TC-05	Test shipper role redirect on user login	Shipper account shipper1@gmail.com exists with role "shipper"	1. Navigate to /login 2. Enter email: shipper1@gmail.com 3. Enter password: 123456 4. Click "Login"	User is automatically redirected to /shipper/dashboard instead of the customer homepage	Redirected to Shipper Dashboard

Menu Browsing & Cart

ID	Test Scenario	Preconditions	Steps to Reproduce	Expected Result	Actual Result
TC-06	Test menu search functionality	Menu page has food items loaded	1. Navigate to /menu 2. Type "burger" in the search bar	Only food items containing "burger" in the name are displayed; non-matching items are hidden	Filtered results displayed correctly

TC-07	Test menu category filter	Menu page has items across multiple categories	1. Navigate to /menu 2. Click on the "Drinks" category filter	Only items in the "Drinks" category are shown; items from other categories are hidden	Category filter works correctly
TC-08	Test add item to cart	User is logged in; menu page is open	1. Navigate to /menu 2. Click "Add to Cart" on a food item 3. Navigate to /cart	The selected item appears in the cart with quantity 1 and correct price	Item added and visible in cart
TC-09	Test update item quantity in cart	User has at least one item in the cart	1. Navigate to /cart 2. Click the "+" button on an item to increase quantity to 3 3. Verify subtotal updates	Quantity updates to 3; subtotal = item price $\times$ 3; cart total recalculates	Quantity and totals updated correctly

Admin dashboard

ID	Test Scenario	Preconditions	Steps to Reproduce	Expected Result	Actual Result
TC-10	Test admin login with correct password	Admin password is admin123	1. Navigate to /admin 2. Enter password: admin123 3. Click "Login"	Admin is authenticated and redirected to the Admin Dashboard	Redirected to Admin Dashboard
TC-11	Test admin advance order status	Admin is logged in; an order exists with	1. Open Admin Dashboard 2. Locate the order in the Orders table	Order status advances from "Confirmed"	Status advanced correctly

		status "Confirmed"	3. Click the "→" (advance) arrow button	(1) to "Preparing" (2); the status label updates in the table	
TC-1 2	Test admin create new shipper account	Admin is logged in	1. Open Admin Dashboard 2. Navigate to the "Shippers" management section 3. Enter shipper name: "New Shipper" 4. Enter email: newshipper@gmail.com 5. Enter password: shipper123 6. Click "Create Shipper"	New shipper account is created in the database with role "shipper"; shipper appears in the shipper list	Shipper account created and listed

Shipper dashboard

ID	Test Scenario	Preconditions	Steps to Reproduce	Expected Result	Actual Result
TC-1 3	Test shipper accept and deliver order	Shipper is logged in; an order with status "Checking" (3) exists; shipper has no current order	1. Open Shipper Dashboard 2. Find an available order with status "Checking" 3. Click "Accept" to pick up the order 4. Order status changes to "Delivering" (4) 5. Click "Delivered"	Order status advances to "Delivering" on accept, then to "Delivered" (5) on delivery; order moves out of active delivery	Order flow completed correctly

			to complete delivery		
TC-1 4	Test shipper cannot hold multiple orders	Shipper is logged in and already has one active delivery	1. Open Shipper Dashboard 2. While holding an active delivery order 3. Attempt to accept another available order	The shipper is prevented from accepting a second order; an error or restriction is enforced	Second order acceptance blocked

6.2 Bug Tracking

During development, we used the GitHub commit history to track and fix bugs. Some major issues included incorrect API routes that caused problems with orders and carts, inconsistent price displays in the menu and incorrect password visibility icons on the registration page. We fixed these issues by updating the API routes, using one price-formatting function, correcting the password icon. GitHub commits and pull requests helped us record each change and review the fixes.

6.3 Known limitations

1. No Real-Time Updates (Polling Instead of WebSockets)

- **Limitation:** The Shipper Dashboard relies on 5-second interval polling to fetch new available orders, which is inefficient and introduces a delay. The customer billing page also requires manual refresh to see status changes.
- **Why:** WebSocket integration (e.g., Socket.IO) requires additional server infrastructure and was outside the initial project scope.
- **Future Plans:** Implement WebSocket connections using Socket.IO to push real-time order status updates to customers, shippers, and admins simultaneously.

2. No Payment Gateway Integration

- **Limitation:** The card payment option only performs basic client-side validation (card starts with "4", 16 digits, 3-digit CVV). No actual payment processing occurs — there is no integration with services like Stripe or PayPal.

- **Why:** Integrating a real payment gateway requires business registration, API keys, and handling sensitive financial data (PCI compliance), which was beyond the scope of an academic project.
- **Future Plans:** Integrate a third-party payment gateway such as Stripe or VNPay for secure, real-world payment processing.

3. Limited Admin Authentication

- **Limitation:** Admin login uses a single shared password (hashed in .env) with no username or multi-admin support. There is no admin account stored in the database, making it impossible to track which admin performed which action.
- **Why:** The system was designed with a single-admin model for simplicity. Role-based admin accounts would require a more complex permission system.
- **Future Plans:** Create a proper admin user model in the database with role-based permissions (e.g., super admin, manager) and add audit logging for admin actions.

CHAPTER 7 DEPLOYMENT

7.1 Deployment Process

The restaurant ordering and table booking web application was deployed as a full-stack web application. The system consists of three main parts: the frontend, the backend API server, and the database. The frontend was developed using React with Vite, while the backend was developed using NestJS. The application was deployed on Railway because it supports web service deployment, environment variable configuration, and hosted database services in one platform.

The deployment process started by preparing the project source code for production. The project was separated into two main folders: one folder for the frontend and one folder for the backend. Each part had its own dependencies, build command, and start command. Before deploying, both parts were tested locally to make sure that the frontend could call the backend API successfully and that the backend could connect to the database.

For the frontend, the React project was built using the production build command:

npm run build

This command generates optimized static files that can be served in production. The frontend was then deployed as a Railway service. Railway automatically detected the Node.js environment and installed the required dependencies from `package.json`. The production build output was served as the live frontend website.

For the backend, the NestJS server was deployed as a separate Railway service. The backend service is responsible for handling API requests such as login, product management, food ordering, table booking, and admin operations. The backend project was configured with a build command and a start command. A common configuration is:

npm install

npm run build

npm run start:prod

The backend server listens to the port provided by Railway through the `PORT` environment variable. This is important because the hosting platform decides which port the application should use. In the NestJS `main.ts` file, the server should listen like this:

await app.listen(process.env.PORT ?? 3000);

This allows the backend to run both locally and on the hosting platform.

After deploying both services, the frontend was connected to the backend by using the deployed backend URL as the API base URL. For example, instead of calling:

http://localhost:3000

the frontend calls the deployed backend URL:

https://endearing-tenderness-production-1ed7.up.railway.app/

This allows users to access the frontend website from the internet while the frontend sends requests to the hosted backend API.

A CORS configuration was also required because the frontend and backend are hosted on different domains. Without CORS, the browser blocks frontend requests to the backend. In the backend, CORS was enabled to allow requests from the deployed frontend URL:

```
app.enableCors({
  origin:
    'https://successful-reverence-production-939c.up.railway.app',
    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE',
    'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization'],
    credentials: true,
});
```

The allowed origin must match the frontend URL exactly. For example, the URL should not contain an extra `/` at the end, because CORS compares origins strictly.

Platform Configuration

The deployment platform used for this project is Railway. Railway was used to host the frontend service, backend service, and database service. The application was divided into separate services so that each part could be managed independently.

The frontend service contains the React application. Its responsibility is to display the user interface and send HTTP requests to the backend API. The backend service contains the NestJS application. Its responsibility is to process business logic, validate requests, authenticate users, and communicate with the database. The database service stores application data such as users, products, orders, bookings, and admin information.

The basic platform configuration is shown below:

Component	Technology	Hosting
Frontend	React + Vite	Railway
Backend	NestJS	Railway

Database	MySQL / PostgreSQL	Railway Database
API Communication	Axios / Fetch API	HTTPS
Authentication	JWT or Cookie-based login	Backend service

Each Railway service has its own settings, deployment logs, and environment variables. This makes the project easier to maintain because frontend errors, backend errors, and database issues can be checked separately.

Environment Variables Setup

Environment variables were used to store configuration values that should not be written directly in the source code. This is important because production values are different from local development values.

For the backend service, the main environment variables include database connection settings, JWT settings, and frontend URL settings.

```
PORT=3000
DATABASE_HOST=mylink
DATABASE_PORT=0000
DATABASE_USER=user
DATABASE_PASSWORD=pass
DATABASE_NAME=restaurant_ordering_system
JWT_SECRET=your-secret-key
JWT_EXPIRES_IN=1d
```

```
FRONTEND_URL=my uilink
```

The backend uses these variables to connect to the hosted database and to configure authentication. The **JWT_SECRET** is especially important because it is used to sign and verify JWT tokens. It should not be exposed publicly.

For the frontend service, the main environment variable is the backend API URL.

Example frontend environment variable:

```
VITE_API_URL=https://restaurant-ordering-system-production-fl3.up.railway.app
```


In React with Vite, environment variables must start with `VITE_` to be accessible inside the frontend code. The frontend can then use this value when making API requests:

```
const API_URL = import.meta.env.VITE_API_URL;
```

This setup makes the project more flexible. During development, the frontend can use `localhost`, while in production it can use the deployed backend URL.

Database Setup on Hosting Platform

The database was created on the hosting platform and connected to the backend service through environment variables. After creating the database service on Railway, Railway provides connection information such as host, port, username, password, and database name.

The backend uses these values to connect to the database. For example, in a NestJS project using TypeORM, the database connection may be configured like this:

```
TypeOrmModule.forRoot({
  type: 'mysql',
  host: process.env.DATABASE_HOST,
  port: Number(process.env.DATABASE_PORT),
  username: process.env.DATABASE_USER,
  password: process.env.DATABASE_PASSWORD,
  database: process.env.DATABASE_NAME,
  entities: [__dirname + '/../**/*.entity{.ts,.js}'],
  synchronize: false,
});
```

In production, `synchronize` should usually be set to `false` to avoid accidentally changing or deleting database tables. Database tables should be created using SQL scripts or migration files.

The database stores the main data of the restaurant system, including:

Data	Description
Users	Customer and admin account information
Products / Food Items	Menu items displayed in the restaurant system
Orders	Customer food orders
Order Details	Food items belonging to each order
Tables	Restaurant table information
Bookings	Customer table reservations
Authentication Data	Login credentials or tokens

After the database was created, the necessary tables were added. Initial sample data may also be inserted to test the system, such as admin accounts, food categories, menu items, and available tables.

The backend was then tested using API requests to confirm that the database connection worked correctly. For example, product listing, login, order creation, and booking creation APIs were tested after deployment.

7.2 Monitoring and Maintenance

After deployment, monitoring and maintenance are necessary to make sure the application continues to run correctly. Since the system is hosted online, errors can happen because of server issues, database connection problems, incorrect environment variables, or invalid user requests.

Application health is monitored mainly through Railway service logs. Railway provides logs for each deployed service, including frontend logs, backend logs, and database-related messages. These logs are useful for checking whether the application started successfully and whether any runtime errors occurred.

For the backend, important things to monitor include:

Monitoring Area	Purpose
------------------------	----------------

Server startup logs	Check whether the backend starts successfully
API request errors	Detect failed API calls
Database connection logs	Check whether the backend can connect to the database
Authentication errors	Detect login or token problems
CORS errors	Check whether the frontend is allowed to call the backend
Deployment logs	Detect build or dependency errors

A simple health check endpoint can also be added to the backend. For example:

```
@Get('/health')
healthCheck() {
  return {
    status: 'ok',
    message: 'Backend is running',
    timestamp: new Date(),
  };
}
```

This endpoint can be opened in the browser or tested using Postman. If the endpoint returns a successful response, it means the backend service is running.

Logging Strategy

The logging strategy focuses on recording important backend events and errors. During development, console logs can be used to debug the application. However, in production, logs should be meaningful and should not expose sensitive information such as passwords, JWT secrets, or database credentials.

The backend should log important events such as:

Server started successfully

Database connected successfully

User login failed
Order created successfully
Booking created successfully
Internal server error

For error handling, the backend should return clear HTTP responses to the frontend.
For example:

Situation	HTTP Status
Successful request	200 OK
Created new data	201 Created
Invalid input	400 Bad Request
Unauthorized login	401 Unauthorized
Forbidden access	403 Forbidden
Data not found	404 Not Found
Server error	500 Internal Server Error

This makes debugging easier because both the frontend and backend can understand what kind of error happened.

The backend should also use validation to prevent invalid data from entering the system. For example, login requests should require an email and password, order requests should require product information, and booking requests should require booking date and table information.

Backup Strategy

The database is the most important part of the system because it stores all user, order, booking, and product data. Therefore, a backup strategy is needed to prevent data loss.

For this project, the backup strategy can include exporting the database regularly. The database can be backed up manually from the hosting platform or by using database

export tools. A backup file can be stored safely so that the database can be restored if data is accidentally deleted or corrupted.

A simple backup strategy is:

Backup Type	Description
Manual backup	Export the database before major updates
Scheduled backup	Create regular database backups if supported by the hosting platform
Pre-deployment backup	Backup before changing database structure
Local copy	Keep a copy of important SQL scripts locally

Before updating the backend or changing the database structure, a backup should be created. This reduces the risk of losing important data during deployment.

Maintenance Plan

The application should be maintained regularly after deployment. Maintenance includes checking logs, fixing bugs, updating dependencies, improving security, and testing important features after each deployment.

The main maintenance activities include:

Maintenance Task	Description
Check deployment logs	Confirm that the latest deployment works
Test main features	Test login, order, booking, and admin functions
Update dependencies	Keep frontend and backend packages updated
Review environment variables	Make sure production values are correct

Monitor database usage	Check whether the database is working correctly
Fix reported bugs	Resolve errors found by users or during testing

For security maintenance, sensitive information must remain inside environment variables. The source code should not contain database passwords, JWT secrets, or private keys. Authentication should also be tested regularly to make sure that users cannot access admin functions without permission.

Overall, the deployment process allows the restaurant ordering and table booking system to be accessed online. The frontend, backend, and database are hosted separately but connected through environment variables and API calls. Monitoring, logging, and backup strategies help keep the application stable, secure, and maintainable after deployment.

CHAPTER 8 USER GUIDE

8.1 Getting Started

8.1.1 How to Create an Account

VnFood supports three user roles, each with its own access method:

Customer Account (Self-Registration)

1. Navigate to the Login page (/login).
2. Click the "Create one" link below the Sign In button.
3. On the Registration page, fill in the required fields:
 - User Name — Letters only (e.g., Nguyen Van A)
 - Email Address — Must be a valid, unique email
 - Password — Minimum 8 characters
 - Confirm Password — Must match the password above
4. Click "Create Account".
5. On success, you will be automatically logged in and redirected to the Home page.

Shipper Account (Admin-Created Only)

- Shipper accounts cannot be self-registered. They must be created by an Admin from the Admin Dashboard

Admin Access

1. Navigate to admin's login page (/admin)
2. Type in admin123 in the password
3. Click "Confirm to access" the admin's dashboard

8.1.2 Test Account Credentials

The following pre-configured accounts can be used for testing:

Role	Email / Access	Password	Login URL
Customer	test@gmail.com	test1234	/login
Shipper	shipper1@gmail.com	123456	/login
Admin	(password only)	admin123	/admin

8.2 Features Walkthrough

8.2.1 Customer Features

Browsing the Menu

Route: /menu

1. Click "Thực Đơn" (Menu) in the navigation bar.
2. The full menu loads with all dishes displayed as cards showing name, image, price, and category.
3. Search: Use the search bar at the top to search dishes by name (e.g., type "Phở").
4. Filter by Category: Click the "Bộ Lọc" (Filter) button to expand the filter panel. Select a category radio button (e.g., Phở & Bún, Com, Bánh Mì, Drink).
5. Filter by Price Range: In the filter panel, select a price range (e.g., Under \$5, \$5–\$10).
6. Switch View Mode: Toggle between Grid view (card layout) and List view using the icons next to the filter button.
7. Pagination: If there are more than 12 dishes, use the page navigation at the bottom to browse additional pages.

Quick-View a Dish

1. On the Menu page, click the quick-view button (eye icon) on any food card.
2. A modal pops up displaying:
 - Full-size dish image

- Dish name, description, and price
 - An "Add to Cart" button
3. Click "Add to Cart" to add the item, or click outside / press the close button to dismiss.

Managing Your Cart

Route: /cart

1. Click the cart icon in the navigation bar (or navigate to /cart).
2. The Cart page displays all added items with:
 - Item image, name, and unit price
 - Quantity controls: Use the – and + buttons to decrease or increase the quantity
 - Remove button: Click the trash icon to remove a single item
3. The Order Summary sidebar shows:
 - Subtotal (sum of all items)
 - Discount (if applicable)
 - Delivery Fee
 - Total
4. Click "Thanh Toán" (Checkout) to proceed, or "Tiếp Tục Mua Sắm" (Continue Shopping) to return to the menu.
5. To remove all items at once, click "Xóa Giỏ Hàng" (Clear Cart) at the bottom.

Checkout & Placing an Order

Route: /checkout

1. From the Cart page, click "Thanh Toán" (Checkout).
2. Fill in the Delivery Information:
 - Số Điện Thoại (Phone Number) — Must start with 0 and be exactly 10 digits
 - Địa Chỉ Giao Hàng (Delivery Address) — Full delivery address
3. Select a Payment Method:
 - Tiền Mặt (Cash) — Pay on delivery (COD)
 - Thẻ (Card) — Enter card details:
 - Card number (must start with 4, 16 digits)
 - Cardholder name
 - Expiry date
 - CVV (3 digits)
4. Review the Order Summary on the right showing item previews, subtotal, discount, delivery fee, and total.

5. Click "Đặt Hàng" (Place Order) to submit.
6. On success, you will be redirected to the Order Success page with your order number.

Tracking Your Orders

Route: /billing

1. Click "Hóa Đơn" (Billing / Orders) in the navigation bar.
2. The page displays all your past and current orders, each showing:
 - Bill ID, status badge, payment status
 - Order date, delivery address, phone number
 - Total amount
3. Progress Tracker: Each active order shows a 5-step visual progress bar:
Đã Xác Nhận (Confirmed) → Đang Chuẩn Bị (Preparing) → Đang Kiểm Tra (Checking) → Đang Giao (Delivering) → Đã Giao (Delivered)
4. Expand Details: Click on any order to expand and see the itemized list of dishes with quantities, prices, discount, delivery fee, and total.
5. Cancel an Order: Click the "Hủy Đơn" (Cancel Order) button to cancel. This is only available if the order status has not yet advanced past Preparing.
6. View Receipt: Click the "Hóa Đơn" (Receipt) link to view the full receipt page for that order.

Viewing a Receipt

Route: /orders/:billId/receipt or /billing/:billId

1. From the Billing page, click the "Hóa Đơn" link on any order.
2. The Receipt page shows a full invoice including:
 - Order ID and date
 - Itemized list of dishes with quantities and prices
 - Subtotal, discount, delivery fee, and final total
 - Payment method and payment status

Booking a Table

Route: /table-booking

1. Click "Đặt Bàn" (Table Booking) in the navigation bar.
2. Fill in the reservation form:
 - Họ Tên (Full Name) — Letters only
 - Số Điện Thoại (Phone) — Must start with 0, exactly 10 digits
 - Số Người (Number of People) — Between 1 and 100

- Số Bàn (Number of Tables) — Between 1 and 50
 - Ngày & Giờ (Date & Time) — Must be a future date, up to 3 months ahead
 - Yêu Cầu Đặc Biệt (Special Requests) — Optional notes (e.g., birthday party, dietary needs)
3. Click "Xác Nhận Đặt Bàn" (Confirm Booking).
 4. On success, a confirmation message appears. You can click "Đặt Bàn Khác" (Book Another Table) to make an additional reservation.

Viewing Promotions

Route: /promotions

1. Click "Promotions" in the navigation bar.
2. Browse the current promotional offers and discounts available at the restaurant.

8.2.2 Admin Features

Admin Login

Route: /admin

1. Navigate to /admin in the browser address bar (this page is not linked in the main navigation).
2. Enter the Admin Password (the plain-text password configured on the server).
3. Click "Access Admin Dashboard".
4. On success, you are redirected to the Admin Dashboard.

Admin Dashboard — Overview & Revenue

Route: /admin/dashboard

Upon login, the dashboard displays four summary metric cards at the top:

Metric	Description
Total Bills	Number of all orders ever placed
Total Revenue	Sum of all order totals
Paid Revenue	Revenue from orders marked as paid
Amount Due	Revenue from unpaid orders

Admin: Managing Orders

1. Below the metrics, the Orders Table shows all customer orders with columns:
 - Bill ID, Customer (phone + address), Total, Status, Actions

2. View Details: Click "View Details" on any row. A detail panel opens on the right showing:
 - Order date, address, phone
 - Itemized dish list with quantities
 - Price breakdown (subtotal, discount, delivery, total)
3. Advance Order Status: Click the green arrow (→) button next to the status badge to move the order to the next status in the pipeline:
Confirmed → Preparing → Checking → Delivering → Delivered → Completed
4. Manually Set Status: In the detail panel, use the "Bill status" dropdown to set any status directly.
5. Mark as Paid: In the detail panel, use the "Payment state" dropdown to toggle between Paid and Unpaid.
6. Click "Refresh" in the header to reload all data.

Admin: Managing Dishes (CRUD)

Route: /admin/dishes

1. From the dashboard header, click "Add Dish" or navigate to /admin/dishes.
2. The Dishes page shows a list of all menu items.
3. Create a new dish:
 - Click the "Add Dish" button
 - Fill in: Name, Price, Category, Description
 - Upload a dish image
 - Click "Save"
4. Edit an existing dish:
 - Click the edit button on a dish row
 - Modify the fields and click "Save"
5. Delete a dish:
 - Click the delete button on a dish row
 - Confirm the deletion

Admin: Managing Shippers

1. On the Admin Dashboard, scroll to the "Quản Lý Shipper" (Shipper Management) section.
2. View existing shippers: A table lists all shipper accounts with their ID, Name, and Email.
3. Create a new shipper account:
 - Click "Tạo Shipper Mới" (Create New Shipper)

- A form expands with three fields:
 - Tên (Name) — The shipper's display name
 - Email — Must be a unique email address
 - Mật khẩu (Password) — The shipper's login password
- Click "Tạo Tài Khoản" (Create Account)
- On success, a green confirmation message appears and the new shipper appears in the table

8.2.3 Shipper Features

Shipper Login

Route: /login (same as customer login)

1. Navigate to the standard Login page (/login).
2. Enter the shipper's Email and Password (provided by the Admin).
3. Click "Sign In".
4. The system detects the account's role is “shipper” and automatically redirects to the Shipper Dashboard (/shipper/dashboard).

Shipper Dashboard — Viewing Available Orders

Route: /shipper/dashboard

1. After login, the Shipper Dashboard loads with the header showing "VNF Ship" branding and the shipper's name.
2. If you have no active delivery, the "Đơn Hàng Đang Chờ" (Pending Orders) section appears:
 - Each order card shows: Order ID, status badge, total amount, delivery address, phone number, and item list
 - The dashboard auto-refreshes every 5 seconds to show new orders
 - You can also click "Làm mới" (Refresh) to manually reload

Shipper: Accepting an Order

1. On the available orders list, find an order you want to deliver.
2. Click "Nhận Đơn" (Accept Order) on the order card.
3. The order is assigned to you. The available orders list disappears and is replaced by the "Đơn Hàng Hiện Tại" (Current Order) section.
4. The current order card shows:
 - Order ID and status
 - A 5-step progress bar (Xác nhận → Chuẩn bị → Sẵn sàng → Đang giao → Đã giao)
 - Delivery address, phone number, total amount, and payment method

- Itemized dish list

Shipper: Releasing / Denying an Order

1. If you need to release an accepted order (before the food is ready), click "Hủy nhận đơn" (Release Order).
2. The order is returned to the available pool for other shippers.
3. You can then accept a different order.

Shipper: Picking Up the Order ("Lấy Hàng")

1. Once the Admin advances the order status to "Đang Kiểm Tra" (Checking), a green notification banner appears on your dashboard:  Món ăn đã sẵn sàng! Đến nhà hàng lấy hàng ngay
2. Click the large green "Lấy Hàng" (Pick Up) button.
3. The order status updates to "Đang Giao" (Delivering).
4. A purple notification banner now shows:  Đang giao hàng. Giao xong hãy nhấn "Đã giao"

Shipper: Marking as Delivered ("Đã Giao")

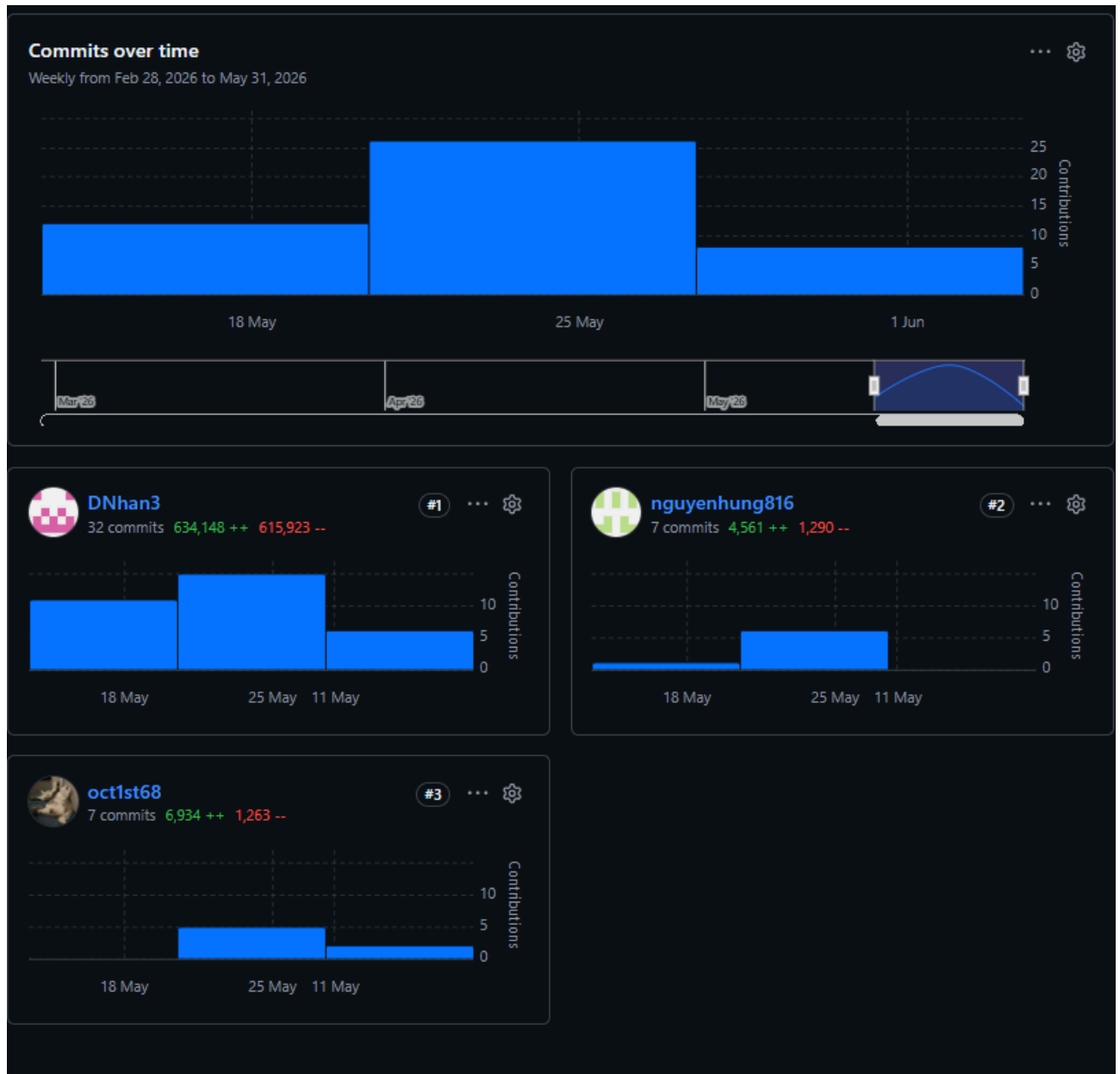
1. After delivering the food to the customer, click the purple "Đã Giao" (Delivered) button.
2. The order status updates to "Đã Giao" (Delivered).
3. A success message appears: "Giao hàng thành công!"
4. The current order section clears and the available orders list reappears, ready for you to accept a new delivery.

CHAPTER 9 TEAM COLLABORATION

9.1 Team Organization

- **Dividing work:**
 - Backend: Nguyễn Triều Hưng , Vũ Đức Nhân
 - Frontend: Phạm Như Nguyên Trung
- **Meeting schedule:** Meeting every week on Saturday or Sunday in order to review the works of each member and give out new tasks
- **Communication tools used:** Facebook's messenger for texting, Discord for meetings and sharing resources

9.2 Individual contributions



9.3 Challenges in Teamwork

- Lack of communication
- Conflict when merging
- Uneven workload
- Lacking knowledge

CHAPTER 10

CONCLUSION AND REFLECTION

10.1 Project Summary

VnFood is a full-stack web-based restaurant ordering and management system developed as part of a Web Application Development course project. Built with React 19 and Vite on the frontend, NestJS 11 with TypeScript on the backend, and MySQL as the database, the application provides a complete digital solution for restaurant operations — from customer ordering to food delivery.

The system supports three distinct user roles, each with a dedicated interface. Customers can browse a categorized food menu with advanced filtering (by category, price range, availability, and food type), search for dishes, and add items to a persistent shopping cart. They can proceed through a validated checkout process with support for both cash and card payment methods, track their orders in real time through a six-stage status pipeline (Confirmed → Preparing → Checking → Delivering → Delivered → Completed), and cancel orders before they reach the kitchen. Customers can also book tables for future dining with date and party-size validation.

Administrators access a comprehensive dashboard featuring key business metrics (total revenue, order counts, customer statistics), a complete order management table with the ability to advance order statuses and toggle payment states, full CRUD operations for managing the food menu (add, edit, delete dishes with image uploads), and a shipper account management panel for creating and overseeing delivery personnel.

Shippers have a dedicated delivery dashboard that polls for available orders in real time, allowing them to accept, pick up, and mark orders as delivered. The system enforces a one-order-at-a-time constraint to ensure delivery accountability, and shippers can deny an order before pickup if needed.

In terms of accomplishments, VnFood successfully implements a complete order lifecycle from placement to delivery, role-based access control with secure authentication using bcrypt password hashing, responsive UI design with Tailwind CSS, and real-time data synchronization through polling. The project demonstrates practical application of modern web development principles including RESTful API

design, client-side state management, form validation, and relational database modeling across six interconnected tables.

10.2 Lessons Learned

After finishing this project, we have complete understanding over all the steps and principles of creating a web application. From frontend to backend to database, frameworks, and testing, this project has taught us from the basic features to the advanced test cases so that we properly make a fully functional website that is ready for market deployment and production.

10.3 Individual Reflections

Nguyễn Trung:

In this project, I worked on the frontend part of the restaurant ordering system and helped build pages like the home page, menu, cart, checkout, table booking, login, admin pages, and shipper dashboard.

I learned that frontend validation is really critical. For instance, the checkout page checks phone number, address, and payment information. Also, The booking page checks date, time, and table availability. Moreover, I understood that the backend must check again because users can change frontend data.

If I did this again, I would make some pages smaller by splitting them into more components. Additionally, I would also make error messages more consistent and add more frontend tests.

This project helped me a lot because I could be familiar with React, routing, forms, API calls, login state, and user experience. Through this project, I learned how the frontend is not only about design, but also about making the system easy and safe for users.

Vũ Đức Nhân:

Working on the VNFood Restaurant Ordering System gave us practical experience in building a complete full-stack web application rather than only separate frontend or backend features. The project required us to think about how customers, administrators, and shippers use the same system from different perspectives. This

helped us understand that a restaurant ordering platform is not just a menu page; it also depends on authentication, cart management, checkout, table booking, order tracking, payment status, and delivery workflow. Through this process, we learned how important it is to design clear user flows before writing code, because small inconsistencies between screens or API routes can quickly affect the whole application.

One of the most valuable lessons was learning how to connect the React frontend with the NestJS backend in a reliable way. At first, some frontend service routes and backend controller routes did not match, which caused features such as orders, bookings, and bill details to fail. Fixing these problems showed us the importance of consistent API naming, shared data structures, and careful testing after each change. We also improved our understanding of how backend services should return data in a format that the frontend can use directly. This reduced extra conversion logic and made the code easier to maintain.

The checkout and order management features were the most meaningful parts of the implementation. They required coordination between multiple database tables, including bill status records and bill detail rows. We learned that transactional thinking is important when creating an order, because the system should not save only part of the order if another step fails. The admin and shipper features also helped us see how the same order data can support different responsibilities: administrators need to manage order progress and users, while shippers need a simple dashboard for delivery processing.

Our teamwork improved throughout the project because each member had to understand how their work affected the rest of the system. Frontend changes depended on backend endpoints, backend changes depended on the database schema, and testing depended on having realistic user accounts and sample data. We also learned to communicate more clearly when naming routes, fields, and statuses. This made integration smoother and reduced repeated debugging.

Overall, this project strengthened our technical skills in React, NestJS, MySQL, REST API design, and application testing. More importantly, it helped us understand the discipline needed to complete a working software product: planning features, connecting components, checking edge cases, and documenting setup instructions. If we continued developing VNFood, we would improve deployment stability, add stronger validation, expand automated tests, and refine the user interface for mobile users. The project was a useful opportunity to apply classroom knowledge to a realistic restaurant ordering system with real user roles and complete business workflows.

Nguyễn Triều Hưng

In this project, I worked mostly as a backend developer but the project did provide invaluable practical experience in building a complete, full-stack web application. Moving beyond isolated frontend or backend features, the project required us to design from the distinct perspectives of customers, administrators, and shippers. This highlighted that a robust ordering platform extends far beyond a simple menu page; it relies on the seamless orchestration of authentication, cart management, table booking, order tracking, and delivery workflows.

A core takeaway from this project was gaining a profound understanding of how APIs operate as the critical bridge between system components. Initially, mismatches between React service routes and NestJS controller endpoints caused failures in core features like order creation and billing. Resolving these issues underscored the necessity of consistent API naming conventions, shared data structures, and rigorous integration testing.

Furthermore, I learned and applied mapping and managing data flows across the entire application stack. I learned to design backend services that return optimized, ready-to-use payloads for the frontend, which significantly eliminated redundant data-transformation logic and streamlined UI rendering.

One of the main features I worked on was the shipper role. I had to redesign the initial SQL schema because the users table didn't have the role attribute and wrote the logic, designing a simple dashboard on both the admin and shipper side.

We also experienced how a unified data model dynamically serves different user roles. While administrators require macro-level overviews to manage users and track overall order progress, shippers need a streamlined, high-efficiency dashboard tailored strictly for delivery execution.

Our team dynamics matured significantly as we learned to collaborate effectively within a highly interdependent architecture. Because frontend features depended on backend endpoints, and backend controllers relied on the MySQL database schema, we established tight feedback loops and synchronized our workflows. We learned to communicate proactively about route naming, status flags, and data fields, which dramatically reduced integration bottlenecks and repetitive debugging sessions.

Ultimately, this project solidified our technical proficiency in React, NestJS, MySQL, and REST API design. More importantly, it taught us the engineering discipline required to deliver a complete software product—from initial user flow planning to edge-case handling. Moving forward, future iterations of VNFood would focus on enhancing deployment stability, implementing stricter data validation, expanding automated test coverage, and optimizing the interface for mobile responsive design.

10.4 Future Enhancements

In the future, We would add online payment, discount codes, and live notification updates so customers can easily check their order status. If the number of users increases. We would improve the database, store menu data in a cache, and use more servers to keep the system fast. We would also add more testing, better error handling, and stronger security to make the system more reliable and safe.